

F.prototype

Kya hai?

Constructor function (F) par ek normal property hoti hai jiska naam "prototype" hai. Jab `new F()` chalta hai, to naye object ka `[[Prototype]] = F.prototype` set ho jaata hai.

javascript

```
let animal = { eats: true };

function Rabbit(name) { this.name = name; }
Rabbit.prototype = animal;

let rabbit = new Rabbit("White");
rabbit.eats; // true
```

Confuse mat hona:

- `F.prototype` = function par ek normal property.
- `[[Prototype]]` = object ka hidden inheritance link.

`F.prototype` sirf `new F` ke waqt use hota hai. Baad me `F.prototype` badal doge to **purane objects** purana prototype hi rakhenge, sirf naye objects naya lenge.

Default `F.prototype` aur `constructor`

Har function ka default prototype ek object hota hai jisme sirf ek property `constructor` hoti hai, jo function ko hi point karti hai.

javascript

```
function Rabbit() {}
Rabbit.prototype.constructor === Rabbit; // true

let rabbit = new Rabbit();
rabbit.constructor === Rabbit; // true (prototype se mila)
```

Iska use: kisi object ke constructor se usi type ka naya object banana (`new rabbit.constructor("Black")`).

Dhyan: `constructor` ki guarantee JS nahi deta

Agar poora prototype overwrite kar diya, to `constructor` gayab ho jaata hai.

javascript

```
Rabbit.prototype = { jumps: true };  
new Rabbit().constructor === Rabbit; // false
```

Bachne ke tareeke:

- Overwrite mat karo, bas property add karo: `Rabbit.prototype.jumps = true;`
- Ya `constructor: Rabbit` khud wapas likh do.

Task se seekhi baatein

- `Rabbit.prototype = {}` baad me karne se **bane hue** rabbit par asar nahi padta.
- `Rabbit.prototype.eats = false` karne se asar padta hai, kyunki object **same reference** hai (copy nahi).
- `delete rabbit.eats` sirf rabbit ka apna property hatata hai. Prototype wali chhui nahi jaati.

Yaad rakho

- `F.prototype` set karo, `new F()` us object ko naye objects ka prototype bana deta hai.
- Value object ya `null` honi chahiye.
- Ye magic sirf constructor function par `new` ke saath chalta hai.